

Where Authorization Stops: Effect-Bound Containment, Independent Evidence and the Limits of Preview

Aditya Pratap Singh — Independent researcher — With Apart Research

Research conducted at the AI Incident Response Sprint (<https://apartresearch.com/sprints/ai-incident-response-sprint-2026-09-11-to-2026-09-13>), September 2026

Abstract

In the incident motivating this sprint, evaluated models escaped through the one network egress their sandbox permitted. A containment standard therefore has to state what a permitted route may *do*, and a third party has to be able to check that claim without the lab's network. We test three candidate controls on synthetic backends: authorize the exact effect at the boundary where it executes; retain backend evidence acquired independently of the actor and broker; and treat a pre-execution preview as evidence only where its fidelity has been shown. Across native Git, a Bubblewrap-isolated HTTP gateway, local Pome API twins and hosted Arga twins, execution-time checks prevented tested violations that upstream approval missed. A Git receive-time gate allowed 0/6 forbidden ref updates versus 3/6 for an equally informed broker preview, and an exact-authority gateway allowed 0/12 HTTP attacks versus 8/12 for endpoint restriction, without losing benign completion. Independent backend tape detected 6/6 executed attacks; intent logs detected 0/6. Our original hypothesis, that adaptive quarantine adds prevention, was only partly supported: selective preview blocked write faults the preview could observe, but every execution-only fault escaped, and three small model-attacker pilots were inconclusive. Adding an execution-time effect gate on the same service path then prevented the execution-only fault that both static and preview missed, while completing the legitimate work — execution-side enforcement, not a quarantine advantage. We release a nine-control checklist with offline acceptance tests, every run including failures, and a reviewer procedure that needs no credentials.

1. Introduction

Sandboxes for capability evaluations usually leave some route open: a package mirror, a proxy, a tool API. According to the sprint materials, the July 2026 incident used exactly such a route, a package-registry cache proxy that was the sandbox's only permitted egress [8]. Restricting *which endpoint* an agent may reach does not constrain *what effect* a request to that endpoint produces. An accurate log of what the agent intended can also disagree with what the backend executed.

We ask a practical containment question: **where should authority be checked, and what evidence lets an outside reviewer confirm that only authorized effects occurred?** Our fixed research question was whether an *adaptive* effect-bound broker (server-side capability enforcement plus dynamic quarantine with selective release) prevents more unauthorized effects than intent logging or static authorization while keeping legitimate work usable. We kept that question fixed, tracked 33 intermediate hypotheses (H1–H7 reconstructed from early runs;

H8 onward recorded before their runs), and retained every failed or incomplete result. The answer split cleanly: server-side enforcement was confirmed, the *quarantine* increment was not — and while testing that gap we designed an execution-time effect gate that delivers the prevention quarantine was reaching for by a different route. That gate is an engineering finding of this work, not a claim we set out to make; we report it as such (contributions 3–4).

Threat model. The actor (a scripted or model-generated client) can alter its requests, client configuration, local files, retries and timing, and can try any route its sandbox allows. It cannot edit server-side policy, hooks or backend state directly. The broker, gateway, backend and host are trusted. A compromised egress service or kernel is out of scope (Section 5).

Our main contributions are:

1. **Execution-time authority beats upstream approval at matched utility.** On native Git and an isolated HTTP path, the same policy enforced where the effect executes blocked every tested substitution that passed an equally informed upstream check, with no reduction in benign completion (Tables 1–2).
2. **Independent backend evidence detects what upstream records cannot.** Backend tape exposed all executed post-gate mutations. Intent logs and gate records still matched the approved request (Table 3).
3. **A negative result on preview-based quarantine, and its execution-side fix.** Selective release helps only when the preview reproduces execution behavior; a fault present only at execution escaped in every local, hosted and separate-process preview study (Table 4). Adding an execution-time effect gate on the same service path prevented that fault while completing the legitimate work, generalizing the Git receive-gate result to a second backend (Table 5). Broadening that gate to confine outbound *data flow* also stopped a read exfiltration that a state-only gate, static authorization and preview all miss (Table 6).
4. **An offline-checkable control checklist.** Nine controls, each with a pass/fail rule, a runnable acceptance test, a measured outcome and a known residual limit, plus a credential-free reviewer procedure (Appendix B).

2. Related Work

Complete mediation [1] and the confused-deputy problem [2] motivate checking every access against specific authority rather than ambient permissions. Our capabilities bind tool, canonical arguments, run and request identity, nonce and expiry, which is a standard object-capability pattern rather than a new primitive. AI Control [3] argues for evaluating safety measures against intentionally subversive models. Our scripted suites follow that red-team framing at small scale but are not frontier-agent evaluations. AgentDojo [4] benchmarks prompt-injection attacks against tool-using agents, and CaMeL [5] enforces information-flow policies over an agent's plan. Both reason about what the agent *intends* to do. We complement them by measuring what the *backend* executed and where a check must sit to constrain it.

The gap we address is verifiability. Existing sandbox guidance says what to restrict, but rarely specifies evidence that lets a third party confirm the restriction held. We contribute acceptance tests that separate *prevented*, *detected after execution* and *missed*, and we demonstrate on real, independently implemented backends (native Git hooks [7], Pome twins [6], hosted Arga twins [9]) where plausible designs fail.

3. Methods

Testbeds. All tests use synthetic fixtures and temporary repositories; no production service or victim data was touched (Figure 1).

- *Native Git* (Git 2.43.0). The permitted effect is one exact old → new object transition on refs/heads/review; production must never change. Attacks use real Git configuration (push redirects, extra refsspecs) and changes to client configuration or HEAD between admission and push. The effect is scored from per-push refs and native reflogs, not exit codes.
- *Isolated HTTP*. A client reads a synthetic report and publishes it as one issue on a local Pome GitHub twin [6]. The actor runs in Bubblewrap [10] mount, PID and network namespaces with dropped capabilities. It can reach only one gateway socket, and never holds backend credentials or signing keys.
- *Pome observer study*. A separate process fetches Pome's own request tape and state and stores hashed receipts, so the broker cannot supply that evidence.
- *Quarantine studies*. A local modeled server (720 cases), two hosted Arga GitHub twins used as execution and preview copies [9], and a separate-process local service with durable operation identities.
- *Combined architecture*. The same separate-process local service, adding a third arm whose service enforces an execution-time effect gate inside the write transaction, so preview-based release and execution-side enforcement are compared under one harness (48 cells). A fourth arm additionally confines the outbound response to each tool's declared output class, tested against an exfiltrating read that returns a secret with no state change (68 cells).

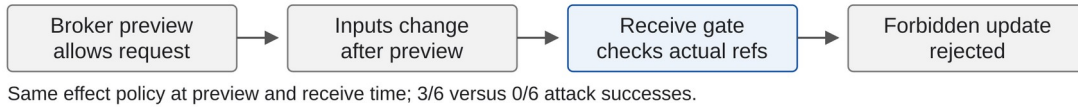
Conditions. Every comparison holds the policy information fixed and varies only *where* or *how* it is enforced. Git: intent-only dispatch, exact command matching, broker native dry-run preview, and a pre-receive hook applying the same effect predicate to Git's actual update batch. HTTP: upstream approval only, sandbox with an endpoint restriction, and sandbox with exact capability revalidation at the gateway. Quarantine: static deny-unknown, hold-all, static authorization of the same reviewed request shapes, and selective release. Selective release runs the request against a preview copy and issues a one-use, server-signed approval bound to the request, state context and policy revision. Combined: static authorization, selective release, and an execution-time effect gate that shares the same authority but additionally confines each operation's committed effect at the service (a read changes no state; a write adds at most its own exact issue), rolling back atomically otherwise — a static effect policy at the effect boundary, the service-path analog of the Git pre-receive hook, not adaptive reasoning.

Scoring. Attack success means an actual unauthorized backend effect: changed refs, a Pome or Arga event or state change, or a witnessed sink delivery. Benign completion checks the correct output and flags duplicates. False rejection, false quarantine, detection and unknown evidence use separate denominators. Missing or malformed evidence is scored *unknown*, never clean.

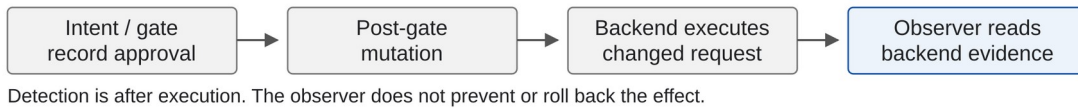
Protocol discipline. From H8 onward, each study's hypotheses and decision rules were recorded before canonical runs (internal timestamps, not external preregistration). Rehearsals were stored separately, and failed prerequisites and unrun cells were kept. Cases are deterministic and constructed to exercise known failure modes, so repetitions are consistency checks rather than independent samples. We report exact counts and make no population or significance claims.

What did not work. Exact state restore was unavailable for both twin providers. Pome seeds do not reproduce timestamps or commit hashes, and Arga resets regenerate Git identifiers. Real-backend quarantine therefore returns an explicit “unsimulated hold” instead of a fabricated clone. Three small model-attacker pilots (GPT-5.4 mini / GPT-5.4, \$1.39 in total) failed to produce substantive attacks (Appendix C).

A. Native Git — enforcement timing



B. Pome — independent observation



C. Local quarantine — preview fidelity

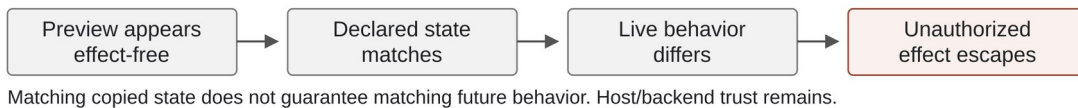


Figure 1. Three tested boundaries. Each row is a separate experiment, not one combined deployment. (A) The same effect policy checked at broker preview versus Git receive time. (B) Post-gate mutations are visible only in independently acquired backend evidence; detection happens after execution. (C) A preview can match declared state yet diverge in live behavior.

4. Results

Table 1. Native Git enforcement (44 cells). Six attack and five benign workflows per condition; identical effect policy in the last two rows. Source: results/git-enforcement-v1.

Condition	Attack successes / 6	Benign completion / 5	False rejection / 5
Intent-only dispatch	6	5	0
Exact command match	6	3	2
Broker native preview	3	5	0
Receive-time effect gate	0	5	0

The three attacks that passed the preview each changed client state *after* admission (push redirect, extra target, wrong commit). The receive gate rejected them because it evaluates the update Git actually receives. Mixed allowed/forbidden pushes left all refs and journals unchanged. The gate also held under missing, malformed and revoked policies, duplicate updates, deletions and -no-verify. Exact command matching is a poor proxy: it rejected two harmless variations and missed all configuration-driven violations. Median benign dispatch was 40.8 ms with the gate versus 12.4 ms intent-only (single-run, descriptive).

Table 2. Isolated file-to-HTTP publishing (51 cells). Twelve attacks (credential file, symlink, direct TCP exfiltration, wrong project, altered content or method, missing token, identity and

nonce rebinding, replay, subprocess and deferred substitution) and five benign workflows. Source: results/isolated-http-v1.

Condition	Attack successes / 12	Correct benign completion / 5	Duplicate benign effects
Upstream approval only	11	4	1
Sandbox + endpoint restriction	8	4	1
Sandbox + exact gateway authority	0	5	0

The sandbox removed the three filesystem and network bypasses. Exact revalidation at the only reachable gateway removed the remaining eight request, identity and replay violations. One unsupported DELETE attack failed natively in every condition and adds no evidence. Weak conditions re-executed a lost-response retry; the exact gateway returned the stored result. Median actor time was about 70 ms in all three conditions.

Table 3. Independent evidence (16 Pome cells). Six executed attacks, two prevented attempts, eight benign controls including 404s and lost responses. Source: results/observer-comparison-v1.

Evidence view	Executed attacks detected / 6	Prevented attempts alerted / 2	Benign false alarms / 8
Agent intent log	0	0	0
Gate-entry request	2	2	0
Backend request tape	6	0	0

Two post-gate substitutions returned seeded file content without changing state, so a state diff alone would also miss them. Detection bounds were 4.5–22.2 ms after dispatch. Detection does not undo an effect: under full enforcement, both post-gate attacks still executed. A later native Git study confirmed the same pattern. Request-text matching raised 4/10 benign false alarms and missed 6/6 configuration-driven violations, while Git's porcelain response flagged all 6.

Table 4. Preview-based quarantine. Top: 720 local cases (five attack and four benign workflows × 20 repetitions). Bottom: fault controls on hosted Arga twins (H26) and a separate-process local service (H29). ✓ = unauthorized effect executed; ✗ = prevented. Sources: results/selective-release-v1, arga-quarantine-v1, local-readiness-v1.

Local modeled (720)	Attack successes / 100	Benign completion / 80	False reject / quarantine
Static deny-unknown	0	0	80 / 0
Hold-all quarantine	0	0	0 / 80
Static, same reviewed shapes	100	80	0 / 0
Selective release	20	80	0 / 0

Fault control	Static (reviewed shapes)	Selective release
Hidden write visible in preview (Arga)	✓	X
Visible / deferred write (H29)	✓ / ✓	X / X
Write that occurs only at execution (Arga, H29)	✓	✓

Selective release is a real but conditional improvement. It matched static authorization on benign reads (80/80; 2/2 on Arga) and blocked every fault its preview could observe. All 20 local simulation-gap repetitions, and the execution-only fault on both later testbeds, escaped and were flagged only after execution. Selective and static both beat blanket denial on utility. Neither beat blanket denial on prevention: an earlier 800-case matrix tied static denial and hold-all at 0/140 attack successes (Appendix C). Selective median latency was 1.33 s versus 0.55 s static in the H30 pilot.

Table 5. Combined architecture (48 cells). The three arms on one service path over the four fault controls; all three complete 12/12 normal cells with no effect and no holds. ✓ = unauthorized effect executed; X = prevented; *hold* = benign read held (utility lost); *infra* = committed write, response lost, reconciliation needed. Source: results/combined-gate-v1.

Fault control	Static	Selective	Effect gate
Hidden write visible in preview	✓	X (hold)	X (completes)
Deferred write	✓	X (hold)	X (completes)
Write only at execution	✓	✓	X (completes)
Response lost after commit	infra	infra	infra

The execution-time effect gate was the only arm to prevent the execution-only write, and the only arm to prevent the preview-visible faults *without* holding the benign read. It does not repair the transport fault: a committed write with a lost response still needs reconciliation in every arm. This is the same execution-side mechanism as the Git receive gate, now on a second backend; it is not a quarantine advantage and does not defend a compromised service or host.

Table 6. Data-flow gate (68 cells). Adding an exfiltrating read — a read that changes no state but returns a secret outside its declared output class, visible only at execution. *state gate* is the Table 5 effect gate; *data-flow gate* also confines the outbound response. *leak* = secret reached the actor; X = prevented. Every arm completes 12/12 normal cells; the data-flow gate matches the state gate on all four Table 5 faults. Source: results/dataflow-gate-v1.

Fault	Static	Selective	State gate	Data-flow gate
Read exfiltration	leak	leak	leak	X (completes)
Write only at execution	✓	✓	X	X

The exfiltrating read exposes the state gate's blind spot: it changes no state, so the state oracle calls it clean and the state gate, static and selective all let the secret through. Confining the *outbound data flow* at execution catches it and still returns the authorized boolean. The gates layer cleanly — data-flow $\supseteq$ state — which is the general point: any effect you can observe and withhold at the execution boundary belongs in the gate, not in a preview that only predicts it. This covers explicit payloads only; covert and implicit channels are out of scope for every arm here.

Robustness. The central contrasts (0 vs 3 of 6; 0 vs 8 of 12; 6 vs 0 of 6) are deterministic under unchanged cases and reproduced in regression tests. They are not rate estimates. The attack sets are purpose-built, which shows that each failure mode *can* occur and that the control stops it, not how often it occurs. All 270 tests pass; `verify_results.py` and the report consistency tests check every table against the raw per-cell archives.

5. Discussion and Limitations

Implications for containment standards. “Only one permitted egress” is not a sufficient requirement. Our results support three requirements a lab could adopt and a reviewer could check offline: (i) the permitted route checks the *exact effect*, meaning request, identity, one-use nonce and target, at the component that executes it; (ii) the lab retains backend-side evidence that the actor and broker cannot write; and (iii) any dry run or preview is treated as advisory unless an execution-side check also constrains the effect. Appendix B turns these into acceptance tests. The cost in our testbeds was tens of milliseconds per action for enforcement and roughly one extra second for preview.

Where the gate ends and quarantine begins. The gate and a preview run the same effect predicate at two different times: the gate on the real effect at execution, the preview on a copy beforehand. So the gate dominates for any effect it can *observe and withhold at its boundary* — state changes (Table 5) and outbound data (Table 6) both qualify, and each was better placed in the gate than predicted on a copy. Quarantine’s irreducible residue is the effect a gate structurally cannot reach: one that is irreversible the instant it executes and routes through a boundary the gate does not mediate — an external side effect, resource consumption, or code whose execution is itself the harm. There the only prevention is to run a disposable copy first, and only as faithfully as that copy reproduces the danger without causing it. In short, quarantine is the fallback for a boundary you cannot gate; the better fix is always to gate the boundary. We did not build that residual case here; it is the next study.

Relation to the incident. We did not reproduce the incident. The incident route was compromised through a zero-day in the egress proxy itself. Our gateway is trusted, so our controls do not stop an attacker who gains code execution inside it. The Git and combined-architecture results point to the more robust placement: enforce the exact effect at the receiving backend, not only in the egress process, so that a route which passed upstream review still cannot commit an unauthorized effect. We did not test that placement against a compromised proxy or backend.

Limitations

- **Trust boundary.** Broker, gateway, observer and backend share one host. Receipts detect later tampering, but a source that fabricates internally consistent evidence from the start passes validation (a retained negative test).
- **Scope.** Protection covers refs, issues and declared state in synthetic twins. It does not cover object uploads before pre-receive, resource exhaustion, covert channels, arbitrary malicious tool code, kernel escapes or multi-host deployments.
- **Fidelity.** Neither twin provider supports exact state restore. Argv equivalence was checked only for issue and access state.
- **Sample.** Constructed, deterministic, small scenario sets. Model-attacker pilots produced no substantive attacks, so realistic AI-attacker resistance is untested.

- **Hypothesis status.** The weak-baseline part is supported. The claim that *quarantine* adds prevention over equally informed static enforcement is conditional and unestablished. Our integrated real-backend workflow ran with an explicit hold instead of a real preview.

Future Work

The combined-architecture study (Table 5) constrains execution-only faults on one service path; the next steps are to test that execution-side enforcement against a *compromised* egress service or backend, not just a trusted one, and to extend the effect predicate beyond a single declared effect class. Run model attackers through an interface where they can influence backend behavior, not only request text. Obtain independent reproduction and a lab's adoption-cost assessment of Appendix B.

6. Conclusion

Containment that names a permitted route but not the effects allowed through it is hard to verify and easy to misuse. On two execution paths, native Git and an isolated HTTP gateway, checking exact authority where the effect executes prevented every tested violation that an equally informed upstream check missed, without costing benign completion. Evidence taken from the backend exposed every executed mismatch that intent and gate logs concealed.

Our adaptive hypothesis fared worse, and the way it failed is the useful part. It pointed at the right problem — catch what upstream approval misses — but the wrong mechanism: preview-based quarantine prevented only the faults its preview could see, and a fault that appears only at execution escaped every preview mechanism we tested. The execution-time effect gate we then designed catches those faults, for both state changes and outbound data, while completing the legitimate work. Right problem, wrong mechanism, better mechanism found. We therefore recommend that containment standards require execution-side enforcement and independent evidence, and that they not accept a clean preview as proof of a safe action.

Code and Data

- **Code repository:** <https://github.com/Aditya-PS-05/effect-bound-containment> (code MIT; report, protocols and result archives CC BY 4.0).
- **Data:** every run, including failed prerequisites and unrun cells, with hashed receipts under `results/` (the combined-architecture study is `results/combined-gate-v1`, the data-flow study `results/dataflow-gate-v1`). Hypotheses H1–H33 and their predeclared protocols are in `hypotheses.md`.
- **Reviewer procedure:** `report/README.md#offline-reviewer-procedure`. No accounts, API keys or hosted services are required.
- **Info-hazard note:** the attack fixtures illustrate generic authorization weaknesses against owned synthetic services and contain no exploit for any real system (Appendix A).

Author Contributions

Aditya Pratap Singh conceived the project, set and challenged the hypotheses, and approved every experiment protocol and model budget. Implementation, test execution and drafting were carried out with AI coding assistants under that direction (see LLM Usage Statement).

References

1. Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE*, 63(9).
<https://web.mit.edu/Saltzer/www/publications/protection/>
2. Hardy, N. (1988). The confused deputy. *ACM SIGOPS Operating Systems Review*, 22(4). <https://doi.org/10.1145/54289.871709>
3. Greenblatt, R., Shlegeris, B., Sachan, K., & Roger, F. (2023). AI control: Improving safety despite intentional subversion. arXiv:2312.06942.
<https://arxiv.org/abs/2312.06942>
4. Debenedetti, E., et al. (2024). AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. arXiv:2406.13352.
<https://arxiv.org/abs/2406.13352>
5. Debenedetti, E., et al. (2025). Defeating prompt injections by design. arXiv:2503.18813.
<https://arxiv.org/abs/2503.18813>
6. Pome contributors. Digital Twins (CLI 0.43.0). <https://github.com/pome-sh/digital-twins>
7. Git project. githooks: pre-receive (Git 2.43.0).
https://git-scm.com/docs/githooks/2.43.0#_pre_receive
8. Apart Research (2026). AI Incident Response Sprint, 11–13 September 2026.
<https://apartresearch.com/sprints/ai-incident-response-sprint-2026-09-11-to-2026-09-13>
9. Arga Labs. Digital twins documentation: GitHub twin reference.
<https://docs.argalabs.com/concepts/twin-reference#github>
10. Bubblewrap contributors. Bubblewrap 0.9.0: unprivileged sandboxing tool.
<https://github.com/containers/bubblewrap>

Appendix A. Limitations and Dual-Use Considerations

Assumptions and what breaks if they fail. (1) *Trusted enforcement point*. If the gateway or hook is compromised, Tables 1–2 say nothing about prevention. Only backend-side evidence remains useful, and only if it is stored off-host. (2) *Trusted backend and observer*. If the backend lies consistently from its first record, Table 3 detection fails. Receipts protect only against later alteration. (3) *Sole route*. If any credential or network path bypasses the gateway, exact authority is moot. Without OS isolation, our boundary audit showed a same-user process reading a fixture credential and bypassing the broker. (4) *Preview fidelity*. If execution behavior differs from preview, selective release admits the effect (Table 4).

Method limits. Deterministic, constructed cases; small scenario diversity; shared implementation authors for harness and scorer; same-host timing; Pome tape and state are

separate, non-atomic reads. Model pilots were budget-limited, and one (H27) stopped at 3/18 cells when hosted quota ran out.

Dual use. The fixtures show how exact-request checks, endpoint restrictions and previews can be bypassed. They are generic authorization weaknesses, not exploits against a product, and target only locally created or owned synthetic services. Temporary credentials are generated inside fixtures. Real deployment logs used as effect evidence may contain sensitive data, so access control and redaction must preserve enough detail to verify effects. We release the suite so defenders can test their own boundaries; it should not be pointed at third-party systems.

Appendix B. Evidence-Bound Containment Checklist

Proposed requirements, not an adopted or certified standard. **PASS** requires the acceptance test plus provenance evidence; **FAIL** means a demonstrated violation; **NOT VERIFIED** means missing evidence. A passing test runner can correctly demonstrate a FAIL through a negative fixture.

Control	Acceptance test (in tests/)	Measured outcome	Residual limit
C1 Dispatch only exact authority (request, identity, nonce, expiry)	test_http_boundary.py	0/12 attacks, 5/5 benign (Table 2)	Gateway/host compromise not covered
C2 Deny alternate actor routes	test_http_boundary.py	Endpoint-only 8/12 vs exact gateway 0/12	Deployment route/credential inventory NOT VERIFIED
C3 One effect per operation across retries	test_runtime_recovery.py, test_local_sandbox.py	12/12 ordinary workflows per arm; malformed commit response loses completion	Needs trusted reconciliation; multi-host untested
C4 Independently acquired call/effect evidence	test_evidence.py, test_observers.py	Backend 6/6 vs gate 2/6 vs intent 0/6	First-record forgery; compromised host
C5 Hold missing previews; retain uncertain commits	test_runtime_recovery.py, test_broker_workflow.py	Unsupported or timed-out previews hold; no automatic re-dispatch	Availability loss; escalation untested
C6 Check useful output; preserve every trial	test_local_sandbox.py, test_adaptive_pilot.py	Duplicates counted as failures; unrun cells retained	No general workload estimate
Q1 Validate preview behavior before claiming safety	test_local_sandbox.py (simulation_gap)	Execution-only write escapes static and preview arms	General preview-safety claim: FAIL
C7 Confine the	test_combined_gate.py	Effect gate	Static effect

Control	Acceptance test (in tests/)	Measured outcome	Residual limit
committed effect at the execution boundary		prevents the execution-only write and all preview-visible faults, completes 12/12 normal cells (Table 5)	policy; no compromised-service or transport-fault coverage
C8 Confine outbound data flow to the declared output class	<code>test_dataflow_gate.py</code>	Data-flow gate prevents the read exfiltration that the state gate, static and preview all leak, and returns the authorized value (Table 6)	Explicit payloads only; covert/implicit channels out of scope

Adopting lab must supply: route and credential inventory for the actor environment, versioned policy and enforcement software, evidence anchors stored outside the actor’s host, and an owner for uncertain commits. Local package verification cannot authenticate those.

Reviewer procedure (summary). On Linux with the pinned environment, run `verify_results.py` and then the two `pytest` commands in `report/README.md`. Expected: all pass, *including* negative fixtures that pass because they observe the violation.

Appendix C. Additional Results and Model Pilots

Local 800-case matrix (7 attack and 3 benign scenarios × 20). Attack successes / 140 and accepted legitimate calls / 60: intent-only 140 / 60; broker-only 100 / 40 (20 false quarantines); static server 0 / 40 (20 false rejections); hold-all 0 / 40 (20 false quarantines). Server enforcement beats weaker baselines; hold-all quarantine ties static denial.

Model-attacker pilots (never pooled; cells run = final cells completed / planned; estimated cost):

Study	Calls	Cost	Cells run	Outcome
H27 hosted (GPT-5.4 mini)	10	\$0.14	3 / 18	Hosted quota exhausted; no matched attack pair. Found a request-ID poisoning availability bug, later fixed (H28).
H30 local (GPT-5.4 mini)	10	\$0.10	18 / 18	9/9 tasks per arm; 4 empty and 2 authorized-only candidates, so no substantive attacks.
H31 adaptive (GPT-5.4)	12	\$1.16	0 / 18	9/12 outputs exhausted token budget on reasoning; search gate failed.

The pilots show that request-only attack interfaces behind prebound identities give a model little room to create the backend behavior mismatch that separates quarantine from static enforcement. Future model evaluations should first demonstrate usable candidates on separate development data.

Hypothesis ledger. H1–H33 with evidence paths and outcomes, including refuted (H7, H12) and unsupported (H2) propositions, are in `hypotheses.md`.

LLM Usage Statement

This project was developed with AI coding assistants, including Claude (Anthropic) through Claude Code. They implemented the harness and tests, ran experiments under protocols the author approved, and drafted this report from the project's evidence files. OpenAI GPT-5.4 mini and GPT-5.4 were used as *experimental subjects* in the attacker pilots (Appendix C), not as writing aids. The same results appear in the project's consolidated review report, whose tables automated checks tie to the raw archives (`verify_results.py`, `tests/test_report.py`); the numbers in this report were also cross-checked against raw per-cell records before submission. **[AUTHOR: before submitting, confirm which sections you rewrote and that you reviewed every claim, state that here, and delete this bracketed note.]**